

FRE6871 R in Finance

Lecture#5, Spring 2026

Jerzy Pawlowski jp3900@nyu.edu

NYU Tandon School of Engineering

March 2, 2026



NYU

**TANDON SCHOOL
OF ENGINEERING**

Brownian Motion

Brownian motion B_T is a stochastic process, with the increments dB_t which are independent and normally distributed, with mean zero and variance equal to dt .

$$dB_t = \xi_t \sqrt{dt}$$

Where the ξ_t are random and independent *innovations* following the standard normal distribution $\phi(0, 1)$, with the expected values: $\mathbb{E}[\xi_t] = 0$, $\mathbb{E}[\xi_t^2] = 1$, and $\mathbb{E}[\xi_{t1}\xi_{t2}] = 0$.

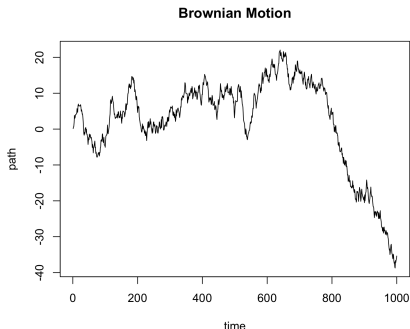
The Brownian motion path B_T is equal to the sum of its increments dB_t :

$$B_T = \sum_{i=1}^n dB_t$$

Where the number of time increments n is equal to the total time of evolution T divided by the increment size dt : $n = T/dt$.

The variance of Brownian motion is equal to the time of its evolution T :

$$\sigma^2 = \mathbb{E}[(\sum_{i=1}^n \xi_i \sqrt{dt})^2] = \sum_{i=1}^n \mathbb{E}[\xi_i^2] dt = T$$



```
> # Simulate a Brownian motion path
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> pathv <- cumsum(rnorm(nrows))
> plot(pathv, type="l", xlab="time", ylab="path",
+       main="Brownian Motion")
```

Geometric Brownian Motion

If the percentage stock returns $r_t dt = d \log p_t$ follow *Brownian motion*:

$$r_t dt = d \log p_t = \left(\mu - \frac{\sigma^2}{2} \right) dt + \sigma dB_t$$

Then asset prices p_t follow *Geometric Brownian motion* (GBM):

$$dp_t = \mu p_t dt + \sigma p_t dB_t$$

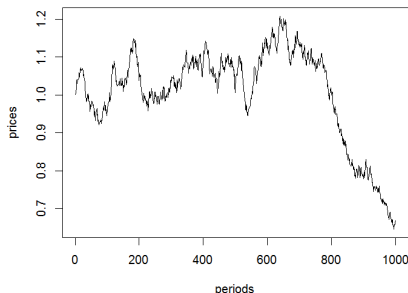
Where σ is the volatility of stock returns, and B_t is a *Brownian Motion*, with dB_t following the normal distribution $\phi(0, \sqrt{dt})$, with the volatility $\sqrt{dt}$, equal to the square root of the time increment dt .

The solution of *Geometric Brownian motion* is equal to:

$$p_t = p_0 \exp\left[\left(\mu - \frac{\sigma^2}{2}\right)t + \sigma B_t\right]$$

The convexity correction: $-\frac{\sigma^2}{2}$ ensures that the growth rate of prices is equal to μ , (according to Ito's lemma).

geometric Brownian motion



```
> # Define the daily volatility and growth rate
> sigmav <- 0.01; drift <- 0.0; nrows <- 1000
> # Simulate geometric Brownian motion
> retp <- sigmav*rnorm(nrows) + drift - sigmav^2/2
> pricev <- exp(cumsum(retp))
> plot(pricev, type="l", xlab="time", ylab="prices",
+      main="Geometric Brownian Motion")
```

The Log-normal Probability Distribution

If x follows the *Normal* distribution $\phi(x, \mu, \sigma)$, then the exponential of x : $y = e^x$ follows the *Log-normal* distribution $\log \phi(\cdot)$:

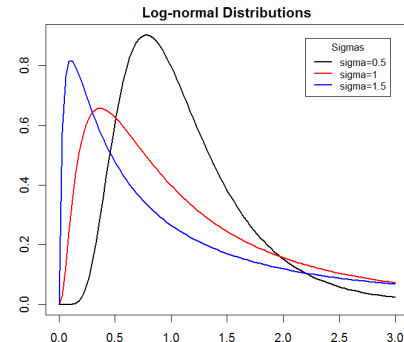
$$\log \phi(y, \mu, \sigma) = \frac{\exp(-(\log y - \mu)^2 / 2\sigma^2)}{y\sigma\sqrt{2\pi}}$$

With mean equal to: $\bar{y} = \mathbb{E}[y] = e^{(\mu + \sigma^2/2)}$, and median equal to: $\tilde{y} = e^\mu$

With variance equal to: $\sigma_y^2 = (e^{\sigma^2} - 1)e^{(2\mu + \sigma^2)}$, and skewness (third moment) equal to:

$$\varsigma = \mathbb{E}[(y - \mathbb{E}[y])^3] = (e^{\sigma^2} + 2)\sqrt{e^{\sigma^2} - 1}$$

```
> # Standard deviations of log-normal distribution
> sigmavs <- c(0.5, 1, 1.5)
> # Create plot colors
> colorv <- c("black", "red", "blue")
> # Plot all curves
> for (indeks in 1:NROW(sigmavs)) {
+   curve(expr=dlnorm(x, sdlog=sigmavs[indeks]),
+         type="l", lwd=2, xlim=c(0, 3),
+         xlab="", ylab="", col=colorv[indeks],
+         add=as.logical(indeks-1))
+ } ## end for
```



```
> # Add title and legend
> title(main="Log-normal Distributions", line=0.5)
> legend("topright", inset=0.05, title="Sigmas",
+       paste("sigma", sigmavs, sep=""),
+       cex=0.8, lwd=2, lty=rep(1, NROW(sigmavs)),
+       col=colorv)
```

The Skewness of Log-normal Prices

If percentage stock returns are *normally* distributed and follow *Brownian motion*, then asset prices follow *Geometric Brownian motion*, and they are *Log-normally* distributed at every point in time.

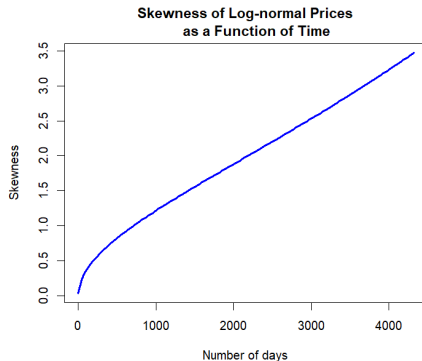
The standard deviation of *log-normal* prices is equal to the return volatility σ_r times the square root of time:
 $\sigma = \sigma_r \sqrt{t}$.

The *Log-normal* distribution has a strong positive skewness (third moment) equal to:

$$\varsigma = \mathbb{E}[(y - \mathbb{E}[y])^3] = (e^{\sigma^2} + 2)\sqrt{e^{\sigma^2} - 1}$$

For large standard deviation, the skewness increases exponentially with the standard deviation and with time: $\varsigma \propto e^{1.5\sigma^2} = e^{1.5t\sigma_r^2}$

```
> # Return volatility of VTI etf
> sigmav <- sd(rutils::diffit(log(rutils::etfenv$VTI[, 4])))
> sigma2 <- sigmav^2
> nrows <- NROW(rutils::etfenv$VTI)
> # Standard deviation of log-normal prices
> sqrt(nrows)*sigmav
```



```
> # Skewness of log-normal prices
> calcskew <- function(t) {
+   expv <- exp(t*sigma2)
+   (expv + 2)*sqrt(expv - 1)
+ } ## end calcskew
> curve(expr=calcskew, xlim=c(1, nrows), lwd=3,
+ xlab="Number of days", ylab="Skewness", col="blue",
+ main="Skewness of Log-normal Prices
+ as a Function of Time")
```

The Mean and Median of *Log-normal* Prices

The mean of the *Log-normal* distribution:
 $\bar{y} = E[y] = \exp(\mu + \sigma^2/2)$ is greater than its median,
 which is equal to: $\tilde{y} = \exp(\mu)$.

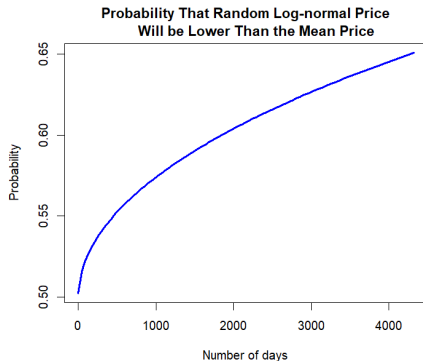
So if stock prices follow *Geometric Brownian motion*
 and are distributed *log-normally*, then a stock selected
 at random will have a high probability of having a lower
 price than the mean expected price.

The cumulative *Log-normal* probability distribution is
 equal to $F(x) = \Phi\left(\frac{\log y - \mu}{\sigma}\right)$, where $\Phi()$ is the
 cumulative standard normal distribution.

So the probability that the price of a randomly selected
 stock will be lower than the mean price is equal to
 $F(\bar{y}) = \Phi(\sigma/2)$.

Therefore an investor without skill, who selects stocks
 at random, has a high probability of underperforming
 the index.

Performing as well as the index requires *significant*
 investment skill, while outperforming the index requires
exceptional investment skill.



```
> # Probability that random log-normal price will be lower than the
> curve(expr=pnorm(sigmav*sqrt(x)/2),
+ xlim=c(1, nrow), lwd=3,
+ xlab="Number of days", ylab="Probability", col="blue",
+ main="Probability That Random Log-normal Price
+ Will be Lower Than the Mean Price")
```

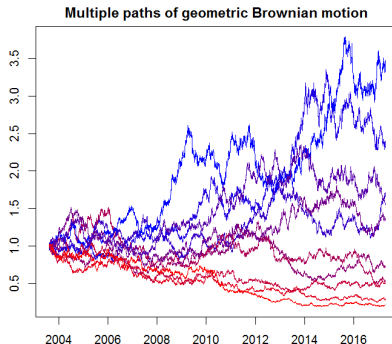
Paths of Geometric Brownian Motion

The standard deviation of *log-normal* prices σ is equal to the volatility of returns σ_r times the square root of time: $\sigma = \sigma_r \sqrt{t}$.

For large standard deviation, the skewness ς increases exponentially with the standard deviation and with

time: $\varsigma \propto e^{1.5\sigma^2} = e^{1.5t\sigma_r^2}$

```
> # Define the daily volatility and growth rate
> sigmav <- 0.01; drift <- 0.0; nrows <- 5000
> npaths <- 10
> # Simulate multiple paths of geometric Brownian motion
> pricev <- rnorm(npaths*nrows, sd=sigmav) + drift - sigmav^2/2
> pricev <- matrix(pricev, nc=npaths)
> pricev <- exp(matrixStats::colCumsums(pricev))
> # Create xts time series
> pricev <- xts(pricev, order.by=seq.Date(Sys.Date()-nrows+1, Sys.Date()),
+               order.by=seq.Date(Sys.Date()-nrows+1, Sys.Date()),
+               order.by=seq.Date(Sys.Date()-nrows+1, Sys.Date()))
> # Sort the columns according to largest terminal values
> pricev <- pricev[, order(pricev[nrows, ])]
> # Plot xts time series
> colorv <- colorRampPalette(c("red", "blue"))(NCOL(pricev))
> par(mar=c(3, 3, 2, 2), oma=c(0, 0, 0, 0))
> plot.zoo(pricev, main="Multiple paths of geometric Brownian motion",
+          xlab=NA, ylab=NA, plot.type="single", col=colorv)
```



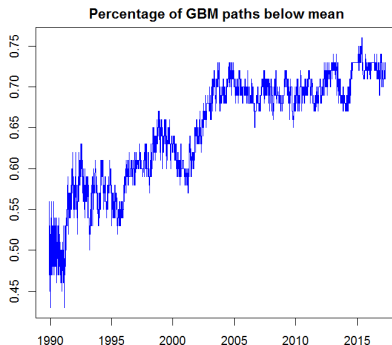
Distribution of Paths of Geometric Brownian Motion

Prices following *Geometric Brownian motion* have a large positive skewness, so that the expected value of prices is skewed by a few paths with very high prices, while the prices of the majority of paths are below their expected value.

For large standard deviation, the skewness ς increases exponentially with the standard deviation and with

$$\text{time: } \varsigma \propto e^{1.5\sigma^2} = e^{1.5t\sigma_r^2}$$

```
> # Define the daily volatility and growth rate
> sigmav <- 0.01; drift <- 0.0; nrows <- 10000
> npaths <- 100
> # Simulate multiple paths of geometric Brownian motion
> pricev <- rnorm(npaths*nrows, sd=sigmav) + drift - sigmav^2/2
> pricev <- matrix(pricev, nc=npaths)
> pricev <- exp(matrixStats::colCumsums(pricev))
> # Calculate fraction of paths below the expected value
> fractv <- rowSums(pricev < 1.0) / npaths
> # Create xts time series of percentage of paths below the expected value
> fractv <- xts(fractv, order.by=seq.Date(Sys.Date()-NROW(fractv)+1, Sys.Date(), by=1))
> # Plot xts time series of percentage of paths below the expected value
> par(mar=c(3, 3, 2, 2), oma=c(0, 0, 0, 0))
> plot.zoo(fractv, main="Percentage of GBM paths below mean",
+         xlab=NA, ylab=NA, col="blue")
```

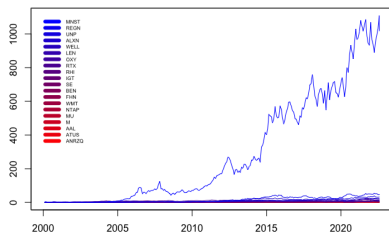


Time Evolution of Stock Prices

Stock prices evolve over time similar to *Geometric Brownian motion*, and they also exhibit a very skewed distribution of prices.

```
> # Load S&P500 stock prices
> load("/Users/jerzy/Develop/lecture_slides/data/sp500.RData")
> ls(sp500env)
> # Extract the closing prices
> pricev <- eapply(sp500env, quantmod::CL)
> # Flatten the prices into a single xts series
> pricev <- rutils::do_call(cbind, pricev)
> # Carry forward and backward non-NA prices
> pricev <- zoo::na.locf(pricev, na.rm=FALSE)
> pricev <- zoo::na.locf(pricev, fromLast=TRUE)
> sum(is.na(pricev))
> # Drop ".Close" from column names
> colnames(pricev)
> colnames(pricev) <- rutils::get_name(colnames(pricev))
> # Or
> # colnames(pricev) <- do.call(rbind,
> #   strsplit(colnames(pricev), split=".["))[, 1]
> # Select prices after the year 2000
> pricev <- pricev["2000/", ]
> # Scale the columns so that prices start at 1
> pricev <- lapply(pricev, function(x) x/as.numeric(x[1]))
> pricev <- rutils::do_call(cbind, pricev)
> # Sort the columns according to the final prices
> nrowv <- NROW(pricev)
> ordern <- order(pricev[nrowv, ])
> pricev <- pricev[, ordern]
> # Select 20 symbols
> symbolv <- colnames(pricev)
> symbolv <- symbolv[round(seq.int(from=1, to=NROW(symbolv), length.out=20))]
```

20 S&P500 Stock Prices (scaled)



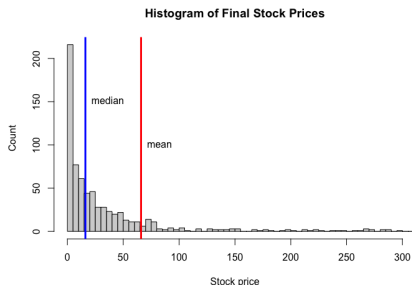
```
> # Plot xts time series of prices
> colorv <- colorRampPalette(c("red", "blue"))(NROW(symbolv))
> endd <- rutils::calc_endpoints(pricev, interval="weeks")
> plot.zoo(pricev[endd, symbolv], main="20 S&P500 Stock Prices (scaled)",
+   xlab=NA, ylab=NA, plot.type="single", col=colorv)
> legend(x="topleft", inset=0.02, cex=0.5, bty="n", y.intersp=0.5,
+   legend=rev(symbolv), col=rev(colorv), lwd=6, lty=1)
```

Distribution of Final Stock Prices

The distribution of the final stock prices is extremely skewed, with over 80% of the *S&P500* constituent stocks from 1990 now below the average price of that portfolio.

The *mean* of the final stock prices is much greater than the *median*.

```
> # Calculate the final stock prices
> pricef <- drop(zoo::coredata(pricev[nrows, ]))
> # Calculate the mean and median stock prices
> max(pricef); min(pricef)
> which.max(pricef)
> which.min(pricef)
> mean(pricef)
> median(pricef)
> # Calculate the percentage of stock prices below the mean
> sum(pricef < mean(pricef))/NROW(pricef)
```



```
> # Plot a histogram of final stock prices
> hist(pricef, breaks=1e3, xlim=c(0, 300),
+       xlab="Stock price", ylab="Count",
+       main="Histogram of Final Stock Prices")
> # Plot a histogram of final stock prices
> abline(v=median(pricef), lwd=3, col="blue")
> text(x=median(pricef), y=150, lab="median", pos=4)
> abline(v=mean(pricef), lwd=3, col="red")
> text(x=mean(pricef), y=100, lab="mean", pos=4)
```

Distribution of Stock Prices Over Time

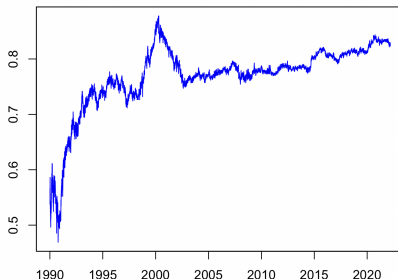
Usually, a small number of stocks in an index reach very high prices, while the prices of the majority of stocks remain below the index price (the average price of the index portfolio).

For example, the current prices of over 80% of the *S&P500* constituent stocks from 1990 are now below the average price of that portfolio.

Therefore an investor without skill, who selects stocks at random, has a high probability of underperforming the index, because they will most likely miss selecting the best performing stocks.

Performing as well as the index requires *significant* investment skill, while outperforming the index requires *exceptional* investment skill.

Percentage of S&P500 Stock Prices Below the Average Price

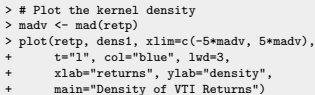


```
> # Calculate average of valid stock prices
> validp <- (pricev != 1) ## Valid stocks
> nstocks <- rowSums(validp)
> nstocks[1] <- NCOL(pricev)
> indeks <- rowSums(pricev*validp)/nstocks
> # Calculate fraction of stock prices below the average price
> fractv <- rowSums((pricev < indeks) & validp)/nstocks
> # Create xts time series of average stock prices
> indeks <- xts(indeks, order.by=zoo::index(pricev))
```

```
> dev.new(width=6, height=4, noRStudioGD=TRUE)
> # x11(width=6, height=4)
> # Plot xts time series of average stock prices
> plot.zoo(indeks, main="Average S&P500 Stock Prices (normalized fr
+   xlab=NA, ylab=NA, col="blue")
> # Create xts time series of percentage of stock prices below the a
> fractv <- xts(fractv, order.by=zoo::index(pricev))
> # Plot percentage of stock prices below the average price
> plot.zoo(fractv[-(1:2)],,
+   main="Percentage of S&P500 Stock Prices
+   Below the Average Price",
+   xlab=NA, ylab=NA, col="blue")
```

The kernel density is proportional to the number of data points close to a given point.

The smoothing kernel $K(x)$ is a symmetric function which decreases with the distance x .

$$d(r) = \sum_{j=1}^n K(r - r_j)$$


A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

Kernel Density Using the Function density()

The function `density()` calculates a kernel estimate of the probability density for a sample of data.

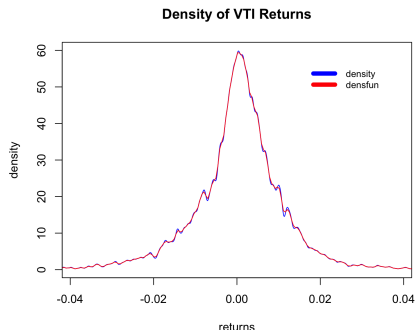
The parameter *smoothing bandwidth* is the standard deviation of the smoothing kernel $K(x)$.

The function `density()` returns a vector of densities at equally spaced points, not for the original data points.

The function `approx()` interpolates a vector of data into another vector.

The function `lines()` draws a line through specified points.

```
> # Calculate the kernel density using density()
> densv <- density(retp, bw=bandwidth)
> NROW(densv$y)
> plot(densv, xlim=c(-5*madv, 5*madv),
+      xlab="returns", ylab="density",
+      col="blue", lwd=3, main="Density of VTI Returns")
> # Interpolate the densv vector into returns
> densv <- approx(densv$x, densv$y, xout=retp)
> all.equal(densv$x, retp)
```



```
> # Plot the two density estimates
> plot(retp, dens1, xlim=c(-5*madv, 5*madv),
+      xlab="returns", ylab="density",
+      t="l", col="blue", lwd=1,
+      main="Density of VTI Returns")
> lines(retp, densv$y, col="red")
> # Add legend
> legend("topright", inset=0.05, cex=0.8, title=NULL,
+      leg=c("density", "densfun"), bty="n", y.intersp=0.4,
+      lwd=6, bg="white", col=c("blue", "red"))
```

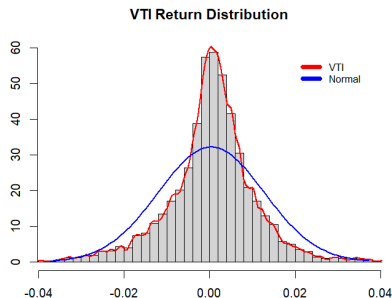
Distribution of Asset Returns

Asset returns are usually not normally distributed and they exhibit *leptokurtosis* (large kurtosis, or fat tails).

The function `hist()` calculates and plots a histogram, and returns its data *invisibly*.

The parameter `breaks` is the number of cells of the histogram.

The function `lines()` draws a line through specified points.



```
> # Plot histogram
> histp <- hist(retp, breaks=100, freq=FALSE,
+   xlim=c(-5*madv, 5*madv), xlab="", ylab="",
+   main="VTI Return Distribution")
> # Draw kernel density of histogram
> lines(densv, col="red", lwd=2)
> # Add density of normal distribution
> curve(expr=dnorm(x, mean=mean(retp), sd=sd(retp)),
+   add=TRUE, lwd=2, col="blue")
> # Add legend
> legend("topright", inset=0.05, cex=0.8, title=NULL,
+   leg=c("VTI", "Normal"), bty="n", y.intersp=0.4,
+   lwd=6, bg="white", col=c("red", "blue"))
```

The Quantile-Quantile Plot

A *Quantile-Quantile (Q-Q)* plot is a plot of points with the same *quantiles*, from two probability distributions.

If the two distributions are similar then all the points in the Q-Q plot lie along the diagonal.

The *VTI Q-Q* plot shows that the *VTI* return distribution has fat tails.

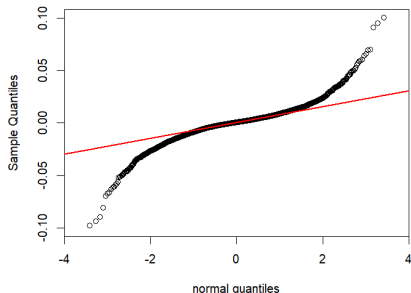
The *p*-value of the *Shapiro-Wilk* test is very close to zero, which shows that the *VTI* returns are very unlikely to be normal.

The function `shapiro.test()` performs the *Shapiro-Wilk* test of normality.

The function `qqnorm()` produces a normal Q-Q plot.

The function `qqline()` fits a line to the normal quantiles.

VTI Q-Q Plot



```
> # Create normal Q-Q plot
> qqnorm(retp, ylim=c(-0.1, 0.1), main="VTI Q-Q Plot",
+   xlab="Normal Quantiles")
> # Fit a line to the normal quantiles
> qqline(retp, col="red", lwd=2)
> # Perform Shapiro-Wilk test
> shapiro.test(retp)
```

Boxplots of Distributions of Values

Box-and-whisker plots (*boxplots*) are graphical representations of a distribution of values.

The bottom and top box edges (*hinges*) are equal to the first and third quartiles, and the *box* width is equal to the interquartile range (*IQR*).

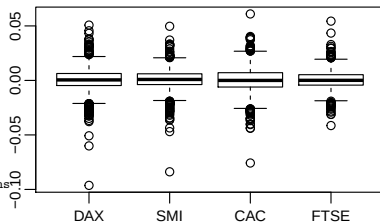
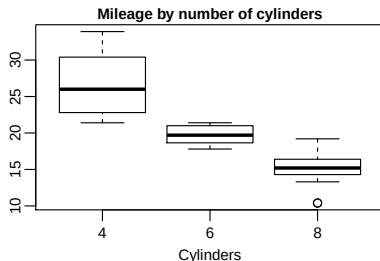
The nominal range is equal to 1.5 times the *IQR* above and below the box *hinges*.

The *whiskers* are dashed vertical lines representing values beyond the first and third quartiles, but within the nominal range.

The *whiskers* end at the last values within the nominal range, while the open circles represent outlier values beyond the nominal range.

The function `boxplot()` has two methods: one for formula objects (for categorical variables), and another for data frames.

```
> # Boxplot method for formula
> boxplot(formula=mpg ~ cyl, data=mtcars,
+   main="Mileage by number of cylinders",
+   xlab="Cylinders", ylab="Miles per gallon")
> # Boxplot method for data frame of EuStockMarkets percentage returns
> boxplot(x=diff(log(EuStockMarkets)))
```



Higher Moments of Asset Returns

The estimators of moments of a probability distribution are given by:

Sample mean: $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$

Sample variance: $\hat{\sigma}^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$

With their expected values equal to the population mean and standard deviation:

$\mathbb{E}[\bar{x}] = \mu$ and $\mathbb{E}[\hat{\sigma}] = \sigma$

The sample skewness (third moment):

$$\varsigma = \frac{n}{(n-1)(n-2)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\hat{\sigma}} \right)^3$$

The sample kurtosis (fourth moment):

$$\kappa = \frac{n(n+1)}{(n-1)(n-2)(n-3)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\hat{\sigma}} \right)^4$$

The normal distribution has skewness equal to 0 and kurtosis equal to 3.

Stock returns typically have negative skewness and kurtosis much greater than 3.

```
> # Calculate VTI percentage returns
> retp <- na.omit(rutils::etfenv$returns$VTI)
> # Number of observations
> nrow <- NROW(retp)
> # Mean of VTI returns
> retm <- mean(retp)
> # Standard deviation of VTI returns
> stdev <- sd(retp)
> # Skewness of VTI returns
> nrow/((nrow-1)*(nrow-2))*sum(((retp - retm)/stdev)^3)
> # Kurtosis of VTI returns
> nrow*(nrow+1)/((nrow-1)^3)*sum(((retp - retm)/stdev)^4)
> # Random normal returns
> retp <- rnorm(nrow, sd=stdev)
> # Mean and standard deviation of random normal returns
> retm <- mean(retp)
> stdev <- sd(retp)
> # Skewness of random normal returns
> nrow/((nrow-1)*(nrow-2))*sum(((retp - retm)/stdev)^3)
> # Kurtosis of random normal returns
> nrow*(nrow+1)/((nrow-1)^3)*sum(((retp - retm)/stdev)^4)
```

Functions for Calculating Skew and Kurtosis

R provides an easy way for users to write functions.

The function `calc_skew()` calculates the skew of returns, and `calc_kurt()` calculates the kurtosis.

Functions return the value of the last expression that is evaluated.

```
> # calc_skew() calculates skew of returns
> calc_skew <- function(retp) {
+   retp <- na.omit(retp)
+   sum(((retp - mean(retp))/sd(retp))^3)/NROW(retp)
+ } ## end calc_skew
> # calc_kurt() calculates kurtosis of returns
> calc_kurt <- function(retp) {
+   retp <- na.omit(retp)
+   sum(((retp - mean(retp))/sd(retp))^4)/NROW(retp)
+ } ## end calc_kurt
> # Calculate skew and kurtosis of VTI returns
> calc_skew(retp)
> calc_kurt(retp)
> # calc_mom() calculates the moments of returns
> calc_mom <- function(retp, moment=3) {
+   retp <- na.omit(retp)
+   sum(((retp - mean(retp))/sd(retp))^moment)/NROW(retp)
+ } ## end calc_mom
> # Calculate skew and kurtosis of VTI returns
> calc_mom(retp, moment=3)
> calc_mom(retp, moment=4)
```

Standard Errors of Estimators

Statistical estimators are functions of samples (which are random variables), and therefore are themselves *random variables*.

The *standard error* (SE) of an estimator is defined as its *standard deviation* (not to be confused with the *population standard deviation* of the underlying random variable).

For example, the *standard error* of the estimator of the mean is equal to:

$$\sigma_{\mu} = \frac{\sigma}{\sqrt{n}}$$

Where σ is the *population standard deviation* (which is usually unknown).

The *estimator* of this *standard error* is equal to:

$$SE_{\mu} = \frac{\hat{\sigma}}{\sqrt{n}}$$

where: $\hat{\sigma}^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$ is the sample standard deviation (the estimator of the population standard deviation).

```
> # Initialize the random number generator
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> # Sample from Standard Normal Distribution
> nrows <- 1000
> datav <- rnorm(nrows)
> # Sample mean
> mean(datav)
> # Sample standard deviation
> sd(datav)
> # Standard error of sample mean
> sd(datav)/sqrt(nrows)
```

Normal (Gaussian) Probability Distribution

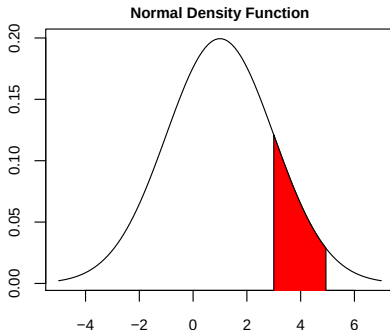
The *Normal (Gaussian)* probability density function is given by:

$$\phi(x, \mu, \sigma) = \frac{e^{-(x-\mu)^2/2\sigma^2}}{\sigma\sqrt{2\pi}}$$

The *Standard Normal* distribution $\phi(0, 1)$ is a special case of the *Normal* $\phi(\mu, \sigma)$ with $\mu = 0$ and $\sigma = 1$.

The function `dnorm()` calculates the *Normal* probability density.

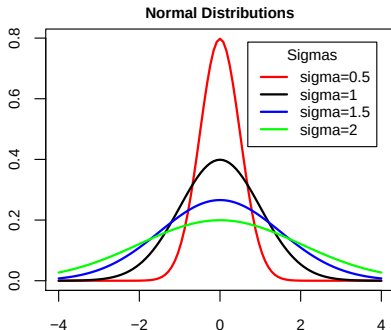
```
> xvar <- seq(-5, 7, length=100)
> yvar <- dnorm(xvar, mean=1.0, sd=2.0)
> plot(xvar, yvar, type="l", lty="solid", xlab="", ylab="")
> title(main="Normal Density Function", line=0.5)
> startp <- 3; endd <- 5 ## Set lower and upper bounds
> # Set polygon base
> subv <- ((xvar >= startp) & (xvar <= endd))
> polygon(c(startp, xvar[subv], endd), ## Draw polygon
+ c(-1, yvar[subv], -1), col="red")
```



Normal (Gaussian) Probability Distributions

Plots of several *Normal* distributions with different values of σ , using the function `curve()` for plotting functions given by their name.

```
> sigmavs <- c(0.5, 1, 1.5, 2) ## Sigma values
> # Create plot colors
> colorv <- c("red", "black", "blue", "green")
> # Create legend labels
> labelv <- paste("sigma", sigmavs, sep="")
> for (it in 1:4) { ## Plot four curves
+   curve(expr=dnorm(x, sd=sigmavs[it]),
+   xlim=c(-4, 4), xlab="", ylab="", lwd=2,
+   col=colorv[it], add=as.logical(it-1))
+ } ## end for
> # Add title
> title(main="Normal Distributions", line=0.5)
> # Add legend
> legend("topright", inset=0.05, title="Sigmas", y.intersp=0.4,
+ labelv, cex=0.8, lwd=2, lty=1, bty="n", col=colorv)
```



Student's t -distribution

Let $z_1, \dots, z_\nu$ be independent standard normal random variables, with sample mean: $\bar{z} = \frac{1}{\nu} \sum_{i=1}^{\nu} z_i$ ($\mathbb{E}[\bar{z}] = \mu$) and sample variance:

$$\hat{\sigma}^2 = \frac{1}{\nu-1} \sum_{i=1}^{\nu} (z_i - \bar{z})^2$$

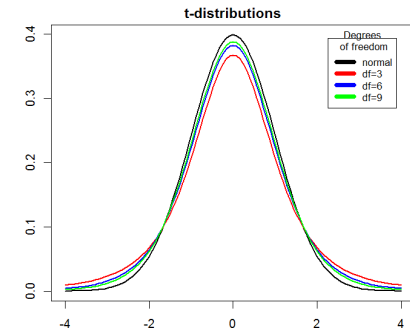
Then the random variable (t -ratio):

$$t = \frac{\bar{z} - \mu}{\hat{\sigma} / \sqrt{\nu}}$$

Follows the t -distribution with ν degrees of freedom, with the probability density function:

$$f(t) = \frac{\Gamma((\nu + 1)/2)}{\sqrt{\pi\nu} \Gamma(\nu/2)} (1 + t^2/\nu)^{-(\nu+1)/2}$$

```
> degf <- c(3, 6, 9) ## Df values
> colorv <- c("black", "red", "blue", "green")
> labelv <- c("normal", paste("df", degf, sep=" "))
> # Plot a Normal probability distribution
> curve(expr=dnorm, xlim=c(-4, 4), xlab="", ylab="", lwd=2)
> for (it in 1:3) { ## Plot three t-distributions
+   curve(expr=dt(x, df=degf[it]), xlab="", ylab="",
+   lwd=2, col=colorv[it+1], add=TRUE)
+ } ## end for
```



```
> # Add title
> title(main="t-distributions", line=0.5)
> # Add legend
> legend("topright", inset=0.05, bty="n", y.intersp=0.4,
+       title="Degrees\n of freedom", labelv,
+       cex=0.8, lwd=6, lty=1, col=colorv)
```

Mixture Models of Returns

Mixture models are produced by randomly sampling data from different distributions.

The mixture of two normal distributions with different variances produces a distribution with *leptokurtosis* (large kurtosis, or fat tails).

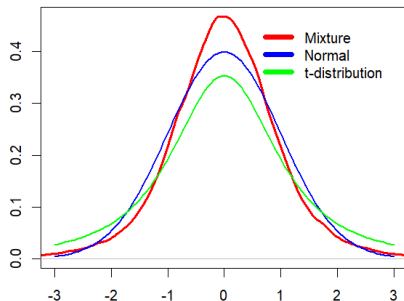
Student's *t-distribution* has fat tails because the sample variance in the denominator of the *t-ratio* is variable.

The time-dependent volatility of asset returns is referred to as *heteroskedasticity*.

Random processes with *heteroskedasticity* can be considered a type of mixture model.

The *heteroskedasticity* produces *leptokurtosis* (large kurtosis, or fat tails).

Mixture of Normal Returns



```
> # Mixture of two normal distributions with sd=1 and sd=2
> nrows <- 1e5
> retp <- c(rnorm(nrows/2), 2*rnorm(nrows/2))
> retp <- (retp-mean(retp))/sd(retp)
> # Kurtosis of normal
> calc_kurt(rnorm(nrows))
> # Kurtosis of mixture
> calc_kurt(retp)
> # Or
> nrows*sum(retp^4)/(nrows-1)^2
```

```
> # Plot the distributions
> plot(density(retp), xlab="", ylab="",
+      main="Mixture of Normal Returns",
+      xlim=c(-3, 3), type="l", lwd=3, col="red")
> curve(expr=dnorm, lwd=2, col="blue", add=TRUE)
> curve(expr=dt(x, df=3), lwd=2, col="green", add=TRUE)
> # Add legend
> legend("topright", inset=0.05, lty=1, lwd=6, bty="n",
+      legend=c("Mixture", "Normal", "t-distribution"), y.intersp=0.4,
+      col=c("red", "blue", "green"))
```

Non-standard Student's *t*-distribution

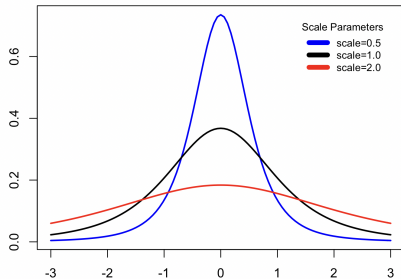
The non-standard Student's *t*-distribution has the probability density function:

$$f(t) = \frac{\Gamma((\nu + 1)/2)}{\sqrt{\pi\nu} \sigma \Gamma(\nu/2)} \left(1 + \left(\frac{t - \mu}{\sigma}\right)^2 / \nu\right)^{-(\nu+1)/2}$$

It has a non-zero mean equal to the location parameter μ , and a standard deviation proportional to the scale parameter σ .

```
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> # x11(width=6, height=5)
> # Define density of non-standard t-distribution
> tdistr <- function(x, dfree, locv=0, scalev=1) {
+   dt((x-locv)/scalev, df=dfree)/scalev
+ } ## end tdistr
> # Or
> tdistr <- function(x, dfree, locv=0, scalev=1) {
+   gamma((dfree+1)/2)/(sqrt(pi*dfree)*gamma(dfree/2)*scalev)*
+   (1+((x-locv)/scalev)^2/dfree)^(-(dfree+1)/2)
+ } ## end tdistr
> # Calculate vector of scale values
> scalev <- c(0.5, 1.0, 2.0)
> colorv <- c("blue", "black", "red")
> labelv <- paste("scale", format(scalev, digits=2), sep="")
> # Plot three t-distributions
> for (it in 1:3) {
+   curve(expr=tdistr(x, dfree=3, scalev=scalev[it]), xlim=c(-3, 3),
+   xlab="", ylab="", lwd=2, col=colorv[it], add=(it>1))
+ } ## end for
```

t-distributions with Different Scale Parameters



```
> # Add title
> title(main="t-distributions with Different Scale Parameters", line=1)
> # Add legend
> legend("topright", inset=0.05, bty="n", title="Scale Parameters",
+       cex=0.8, lwd=6, lty=1, col=colorv, y.intersp=0.4)
```


The *Shapiro-Wilk* Test of Normality

The *Shapiro-Wilk* test is designed to test the *null hypothesis* that a sample: $\{x_1, \dots, x_n\}$ is from a normally distributed population.

The test statistic is equal to:

$$W = \frac{(\sum_{i=1}^n a_i x_{(i)})^2}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Where the: $\{a_1, \dots, a_n\}$ are proportional to the *order statistics* of random variables from the normal distribution.

$x_{(k)}$ is the *k*-th *order statistic*, and is equal to the *k*-th smallest value in the sample: $\{x_1, \dots, x_n\}$.

The *Shapiro-Wilk* statistic follows its own distribution, and is less than or equal to 1.

The *Shapiro-Wilk* statistic is close to 1 for samples from normal distributions.

The *p*-value for *VTI* returns is extremely small, and we conclude that the *null hypothesis* is FALSE, and the *VTI* returns are not from a normally distributed population.

The *Shapiro-Wilk* test is not reliable for large sample sizes, so it's limited to less than 5000 sample size.

```
> # Calculate VTI percentage returns
> library(rutils)
> retp <- as.numeric(na.omit(rutils::etfenv$returns$VTI))[1:499]
> # Reduce number of output digits
> ndigits <- options(digits=5)
> # Shapiro-Wilk test for normal distribution
> nrows <- NROW(retp)
> shapiro.test(rnorm(nrows))
```

Shapiro-Wilk normality test

```
data:  rnorm(nrows)
W = 0.994, p-value = 0.04
> # Shapiro-Wilk test for VTI returns
> shapiro.test(retp)
```

Shapiro-Wilk normality test

```
data:  retp
W = 0.991, p-value = 0.0029
> # Shapiro-Wilk test for uniform distribution
> shapiro.test(runif(nrows))
```

Shapiro-Wilk normality test

```
data:  runif(nrows)
W = 0.964, p-value = 1.2e-09
> # Restore output digits
> options(digits=ndigits$digits)
```

The Jarque-Bera Test of Normality

The *Jarque-Bera* test is designed to test the *null hypothesis* that a sample: $\{x_1, \dots, x_n\}$ is from a normally distributed population.

The test statistic is equal to:

$$JB = \frac{n}{6}(\varsigma^2 + \frac{1}{4}(\kappa - 3)^2)$$

Where the *skewness* and *kurtosis* are defined as:

$$\varsigma = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\hat{\sigma}} \right)^3 \quad \kappa = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\hat{\sigma}} \right)^4$$

The *Jarque-Bera* statistic asymptotically follows the *chi-squared* distribution with 2 degrees of freedom.

The *Jarque-Bera* statistic is small for samples from normal distributions.

The *p*-value for *VTI* returns is extremely small, and we conclude that the *null hypothesis* is FALSE, and the *VTI* returns are not from a normally distributed population.

```
> library(tseries) ## Load package tseries
> # Jarque-Bera test for normal distribution
> jarque.bera.test(rnorm(nrows))
```

Jarque Bera Test

```
data:  rnorm(nrows)
X-squared = 0.2, df = 2, p-value = 0.9
> # Jarque-Bera test for VTI returns
> jarque.bera.test(retp)
```

Jarque Bera Test

```
data:  retp
X-squared = 22, df = 2, p-value = 2e-05
> # Jarque-Bera test for uniform distribution
> jarque.bera.test(runif(NROW(retp)))
```

Jarque Bera Test

```
data:  runif(NROW(retp))
X-squared = 32, df = 2, p-value = 1e-07
```

The Kolmogorov-Smirnov Test for Probability Distributions

The *Kolmogorov-Smirnov* test *null hypothesis* is that two samples: $\{x_1, \dots, x_n\}$ and $\{y_1, \dots, y_n\}$ were obtained from the same probability distribution.

The *Kolmogorov-Smirnov* statistic depends on the maximum difference between two empirical cumulative distribution functions (cumulative frequencies):

$$D = \sup_i |P(x_i) - P(y_i)|$$

The function `ks.test()` performs the *Kolmogorov-Smirnov* test and returns the statistic and its *p*-value *invisibly*.

The second argument to `ks.test()` can be either a numeric vector of data values, or a name of a cumulative distribution function.

The *Kolmogorov-Smirnov* test can be used as a *goodness of fit* test, to test if a set of observations fits a probability distribution.

```
> # KS test for normal distribution
> kstest <- ks.test(rnorm(100), pnorm)
> kstest$p.value
> # KS test for uniform distribution
> ks.test(runif(100), pnorm)
> # KS test for two shifted normal distributions
> ks.test(rnorm(100), rnorm(100, mean=0.1))
> ks.test(rnorm(100), rnorm(100, mean=1.0))
> # KS test for two different normal distributions
> ks.test(rnorm(100), rnorm(100, sd=2.0))
> # KS test for VTI returns vs normal distribution
> retp <- as.numeric(na.omit(rutils::etfenv$returns$VTI))
> retp <- (retp - mean(retp))/sd(retp)
> ks.test(retp, pnorm)
```

Chi-squared Distribution

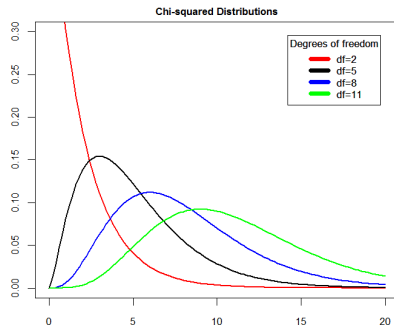
Let $z_1, \dots, z_k$ be independent standard *Normal* random variables.

Then the random variable $X = \sum_{i=1}^k z_i^2$ is distributed according to the *Chi-squared* distribution with k degrees of freedom: $X \sim \chi_k^2$, and its probability density function is given by:

$$f(x) = \frac{x^{k/2-1} e^{-x/2}}{2^{k/2} \Gamma(k/2)}$$

The *Chi-squared* distribution with k degrees of freedom has mean equal to k and variance equal to $2k$.

```
> # Degrees of freedom
> degf <- c(2, 5, 8, 11)
> # Plot four curves in loop
> colorv <- c("red", "black", "blue", "green")
> for (it in 1:4) {
+   curve(expr=dchisq(x, df=degf[it]),
+         xlim=c(0, 20), ylim=c(0, 0.3),
+         xlab="", ylab="", col=colorv[it],
+         lwd=2, add=as.logical(it-1))
+ } ## end for
```



```
> # Add title
> title(main="Chi-squared Distributions", line=0.5)
> # Add legend
> labelv <- paste("df", degf, sep="=")
> legend("topright", inset=0.05, bty="n", y.intersp=0.4,
+       title="Degrees of freedom", labelv,
+       cex=0.8, lwd=6, lty=1, col=colorv)
```

The *Chi-squared* Test for the Goodness of Fit

Goodness of Fit tests are designed to test if a set of observations fits an assumed theoretical probability distribution.

The *Chi-squared* test tests if a frequency of counts fits the specified distribution.

The *Chi-squared* statistic is the sum of squared differences between the observed frequencies o_i and the theoretical frequencies p_i :

$$\chi^2 = N \sum_{i=1}^n \frac{(o_i - p_i)^2}{p_i}$$

Where N is the total number of observations.

The *null hypothesis* is that the observed frequencies are consistent with the theoretical distribution.

The function `chisq.test()` performs the *Chi-squared* test and returns the statistic and its *p*-value *invisibly*.

The parameter `breaks` in the function `hist()` should be chosen large enough to capture the shape of the frequency distribution.

```
> # Observed frequencies from random normal data
> histp <- hist(rnorm(1e3, mean=0), breaks=100, plot=FALSE)
> countsn <- histp$counts
> # Theoretical frequencies
> countst <- rutils::diffit(pnorm(histp$breaks))
> # Perform Chi-squared test for normal data
> chisq.test(x=countsn, p=countst, rescale.p=TRUE, simulate.p.value=TRUE)
> # Return p-value
> chisqtest <- chisq.test(x=countsn, p=countst, rescale.p=TRUE, simulate.p.value=TRUE)
> chisqtest$p.value
> # Observed frequencies from shifted normal data
> histp <- hist(rnorm(1e3, mean=2), breaks=100, plot=FALSE)
> countsn <- histp$counts/sum(histp$counts)
> # Theoretical frequencies
> countst <- rutils::diffit(pnorm(histp$breaks))
> # Perform Chi-squared test for shifted normal data
> chisq.test(x=countsn, p=countst, rescale.p=TRUE, simulate.p.value=TRUE)
> # Calculate histogram of VTI returns
> histp <- hist(retp, breaks=100, plot=FALSE)
> countsn <- histp$counts
> # Calculate cumulative probabilities and then difference them
> countst <- pt((histp$breaks-locv)/scalev, df=2)
> countst <- rutils::diffit(countst)
> # Perform Chi-squared test for VTI returns
> chisq.test(x=countsn, p=countst, rescale.p=TRUE, simulate.p.value=TRUE)
```

The Likelihood Function of Student's *t*-distribution

The non-standard Student's *t*-distribution is:

$$f(t) = \frac{\Gamma((\nu + 1)/2)}{\sqrt{\pi\nu} \sigma \Gamma(\nu/2)} \left(1 + \left(\frac{t - \mu}{\sigma}\right)^2 / \nu\right)^{-(\nu+1)/2}$$

It has non-zero mean equal to the location parameter μ , and a standard deviation proportional to the scale parameter σ .

The negative logarithm of the probability density is equal to:

$$-\log(f(t|\mu, \sigma)) = -\log\left(\frac{\Gamma((\nu + 1)/2)}{\sqrt{\pi\nu} \Gamma(\nu/2)}\right) + \log(\sigma) + \frac{\nu + 1}{2} \log\left(1 + \left(\frac{t - \mu}{\sigma}\right)^2 / \nu\right)$$

The *likelihood* function $\mathcal{L}(\mu, \sigma|\mathbf{t})$ is the product of the individual probability density functions $f(t_i|\mu, \sigma)$:

$$\mathcal{L}(\mu, \sigma|\mathbf{t}) = \prod_{i=1}^n f(t_i|\mu, \sigma)$$

The *likelihood* $\mathcal{L}(\mu, \sigma|\mathbf{t})$ is a function of the model parameters μ and σ , given the vector of the observed values $\mathbf{t}$, under the model's probability distribution $f(t|\mu, \sigma)$:

```
> # Objective function from function dt()
> likefun <- function(par, dfree, datav) {
+   -sum(log(dt(x=(datav-par[1])/par[2], df=dfree)/par[2]))
+ } ## end likefun
> # Demonstrate equivalence with log(dt())
> likefun(c(1, 0.5), 2, 2:5)
> -sum(log(dt(x=2:5-1)/0.5, df=2)/0.5))
> # Objective function is negative log-likelihood
> likefun <- function(par, dfree, datav) {
+   sum(-log(gamma((dfree+1)/2)/(sqrt(pi*dfree)*gamma(dfree/2))) +
+     log(par[2]) + (dfree+1)*2*log(1+((datav-par[1])/par[2])^2/dfree))
+ } ## end likefun
```

The *likelihood* function measures how *likely* are the model parameters μ, σ , given the observed values $\mathbf{t}$.

The *maximum-likelihood* estimate (MLE) of the parameters $\theta = (\mu, \sigma)$ are those that maximize the *likelihood* function:

$$\theta_{MLE} = \arg \max_{\theta} \mathcal{L}(\theta|\mathbf{t})$$

In practice the logarithm of the *likelihood* $\log(\mathcal{L})$ is maximized, instead of the *likelihood* itself.

Fitting Asset Returns into Student's t -distribution

The function `fitdistr()` from package *MASS* fits a univariate distribution to a sample of data, by performing *maximum likelihood* optimization.

The function `fitdistr()` performs a *maximum likelihood* optimization to find the non-standardized Student's t -distribution location and scale parameters.

```
> # Calculate VTI percentage returns
> retp <- as.numeric(na.omit(rutils::etfenv$returns$VTI))
> # Fit VTI returns using MASS::fitdistr()
> fitobj <- MASS::fitdistr(retp, densfun="t", df=3)
> summary(fitobj)
> # Fitted parameters
> fitobj$estimate
> locv <- fitobj$estimate[1]
> scalev <- fitobj$estimate[2]
> locv; scalev
> # Standard errors of parameters
> fitobj$sed
> # Log-likelihood value
> fitobj$value
> # Fit distribution using optim()
> initp <- c(mean=0, scale=0.01) ## Initial parameters
> fitobj <- optim(par=initp,
+   fn=likefun, ## Log-likelihood function
+   datav=retp,
+   dfree=3, ## Degrees of freedom
+   method="L-BFGS-B", ## Quasi-Newton method
+   upper=c(1, 0.1), ## Upper constraint
+   lower=c(-1, 1e-7)) ## Lower constraint
> # Optimal parameters
> locv <- fitobj$par["mean"]
> scalev <- fitobj$par["scale"]
> locv; scalev
```

The Student's t -distribution Fitted to Asset Returns

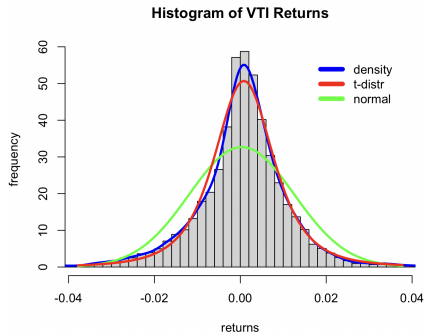
Asset returns typically exhibit *negative skewness* and *large kurtosis* (leptokurtosis), or fat tails.

Stock returns fit the non-standard t -distribution with 3 degrees of freedom quite well.

The function `hist()` calculates and plots a histogram, and returns its data *invisibly*.

The parameter `breaks` is the number of cells of the histogram.

```
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> # x11(width=6, height=5)
> # Plot histogram of VTI returns
> madv <- mad(retp)
> histp <- hist(retp, col="lightgrey",
+   xlab="returns", breaks=100, xlim=c(-5*madv, 5*madv),
+   ylab="frequency", freq=FALSE, main="Histogram of VTI Returns")
> lines(density(retp, adjust=1.5), lwd=3, col="blue")
> # Plot the Normal probability distribution
> curve(expr=dnorm(x, mean=mean(retp),
+   sd=sd(retp)), add=TRUE, lwd=3, col="green")
> # Define non-standard t-distribution
> tdistr <- function(x, dfree, locv=0, scalev=1) {
+   dt((x-locv)/scalev, df=dfree)/scalev
+ } ## end tdistr
> # Plot t-distribution function
> curve(expr=tdistr(x, dfree=3, locv=locv, scalev=scalev), col="red", lwd=3, add=TRUE)
> # Add legend
> legend("topright", inset=0.05, bty="n", y.intersp=0.4,
+   leg=c("density", "t-distr", "normal"),
+   lwd=6, lty=1, col=c("blue", "red", "green"))
```



Goodness of Fit of Student's t -distribution Fitted to Asset Returns

The Q-Q plot illustrates the relative distributions of two samples of data.

The Q-Q plot shows that stock returns fit the non-standard t -distribution with 3 degrees of freedom quite well.

The function `qqplot()` produces a Q-Q plot for two samples of data.

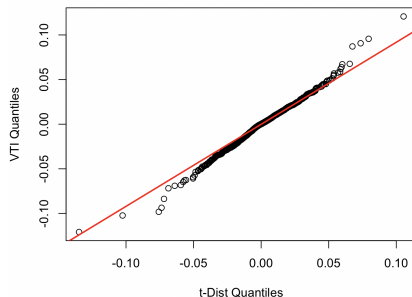
The function `ks.test()` performs the Kolmogorov-Smirnov test for the similarity of two distributions.

The null hypothesis of the Kolmogorov-Smirnov test is that the two samples were obtained from the same probability distribution.

The Kolmogorov-Smirnov test rejects the null hypothesis that stock returns follow closely the non-standard t -distribution with 3 degrees of freedom.

```
> # Calculate sample from non-standard t-distribution with df=3
> datat <- locv + scalev*rt(NROW(retp), df=3)
> # KS test for VTI returns vs t-distribution data
> ks.test(retp, datat)
```

Q-Q plot of VTI Returns vs Student's t -distribution



```
> # Q-Q plot of VTI Returns vs non-standard t-distribution
> qqplot(datat, retp, xlab="t-Dist Quantiles", ylab="VTI Quantiles",
+       main="Q-Q plot of VTI Returns vs Student's t-distribution")
> # Calculate quantiles of the distributions
> probs <- c(0.25, 0.75)
> qrets <- quantile(retp, probs)
> qtdata <- quantile(datat, probs)
> # Calculate slope and plot line connecting quantiles
> slope <- diff(qrets)/diff(qtdata)
> intercept <- qrets[1]-slope*qtdata[1]
> abline(intercept, slope, lwd=2, col="red")
```

Leptokurtosis Fat Tails of Asset Returns

The probability under the *normal* distribution decreases exponentially for large values of x :

$$\phi(x) \propto e^{-x^2/2\sigma^2} \quad (\text{as } |x| \rightarrow \infty)$$

This is because a normal variable can be thought of as the sum of a large number of independent binomial variables of equal size.

So large values are produced only when all the contributing binomial variables are of the same sign, which is very improbable, so it produces extremely low tail probabilities (thin tails),

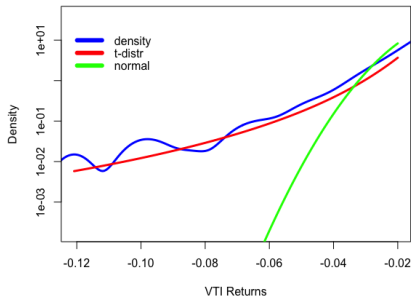
But in reality, the probability of large negative asset returns decreases much slower, as the negative power of the returns (fat tails).

The probability under Student's *t-distribution* decreases as a power for large values of x :

$$f(x) \propto |x|^{-(\nu+1)} \quad (\text{as } |x| \rightarrow \infty)$$

This is because a *t-variable* can be thought of as the sum of normal variables with different volatilities (different sizes).

Fat Left Tail of VTI Returns (density in log scale)



```
> # Plot log density of VTI returns
> plot(density(retp, adjust=4), xlab="VTI Returns", ylab="Density",
+      main="Fat Left Tail of VTI Returns (density in log scale)",
+      type="l", lwd=3, col="blue", xlim=c(min(retp), -0.02), log="y")
> # Plot t-distribution function
> curve(expr=dt((x-locv)/scalev, df=3)/scalev, lwd=3, col="red", add=TRUE)
> # Plot the Normal probability distribution
> curve(expr=dnorm(x, mean=mean(retp), sd=sd(retp)), lwd=3, col="green", add=TRUE)
> # Add legend
> legend("topleft", inset=0.01, bty="n", y.intersp=c(0.25, 0.25, 0.25),
+       legend=c("density", "t-distr", "normal"), y.intersp=0.4,
+       lwd=6, lty=1, col=c("blue", "red", "green"))
```

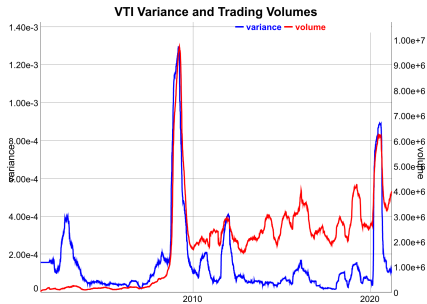
Trading Volumes

The average trading volumes have increased significantly since the 2008 crisis, mostly because of high frequency trading (HFT).

Higher levels of volatility coincide with higher *trading volumes*.

The time-dependent volatility of asset returns (*heteroskedasticity*) produces their fat tails (*leptokurtosis*).

```
> # Calculate VTI returns and trading volumes
> ohlc <- rutils::etfenv$VTI
> closep <- drop(coredata(quantmod::Cl(ohlc)))
> retp <- rutils::diffit(log(closep))
> volumv <- coredata(quantmod::Vo(ohlc))
> # Calculate trailing variance
> lookb <- 121
> varv <- HighFreq::roll_var_ohlc(log(ohlc), method="close", lookb=lookb, scale=FALSE)
> varv[1:lookb, ] <- varv[lookb+1, ]
> # Calculate trailing average volume
> volumr <- HighFreq::roll_sum(volumv, lookb=lookb)/lookb
> # dygraph plot of VTI variance and trading volumes
> datav <- xts::xts(cbind(varv, volumr), zoo::index(ohlc))
> colv <- c("variance", "volume")
> colnames(datav) <- colv
> dygraphs::dygraph(datav, main="VTI Variance and Trading Volumes") %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], strokeWidth=2, axis="y", col="blue") %>%
+   dySeries(name=colv[2], strokeWidth=2, axis="y2", col="red") %>%
+   dyLegend(show="always", width=500)
```



Asset Returns in Trading Time

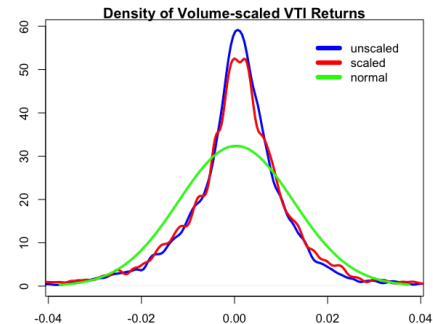
The time-dependent volatility of asset returns (*heteroskedasticity*) produces their fat tails (*leptokurtosis*).

If asset returns were measured at fixed intervals of *trading volumes* (*trading time* instead of clock time), then the volatility would be lower and less time-dependent.

The asset returns can be adjusted to *trading time* by dividing them by the *square root of the trading volumes*, to obtain scaled returns over equal trading volumes.

The scaled returns have a more positive *skewness* and a smaller *kurtosis* than unscaled returns.

```
> # Scale the returns using volume clock to trading time
> retsc <- ifelse(volumv > 0, sqrt(volumv)*retp/sqrt(volumv), 0)
> retsc <- sd(retp)*retsc/sd(retsc)
> # retsc <- ifelse(volumv > 1e4, retp/volumv, 0)
> # Calculate moments of scaled returns
> nrow <- NROW(retp)
> sapply(list(retp=retp, retsc=retsc),
+   function(rets) {sapply(c(skew=3, kurt=4),
+     function(x) sum((rets/sd(rets))^x)/nrow)
+ }) ## end sapply
```



```
> # x11(width=6, height=5)
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> par(mar=c(3, 3, 2, 1), oma=c(1, 1, 1, 1))
> # Plot densities of SPY returns
> madv <- mad(retp)
> # bwld <- mad(rutils::diffit(retp))
> plot(density(retp, bw=madv/10), xlim=c(-5*madv, 5*madv),
+   lwd=3, mgp=c(2, 1, 0), col="blue",
+   xlab="returns (standardized)", ylab="frequency",
+   main="Density of Volume-scaled VTI Returns")
> lines(density(retsc, bw=madv/10), lwd=3, col="red")
> curve(expr=dnorm(x, mean=mean(retp), sd=sd(retp)),
+   add=TRUE, lwd=3, col="green")
> # Add legend
> legend("topright", inset=0.05, bty="n", y.intersp=0.4
```

Trade and Quote (TAQ) Data

High frequency data is typically formatted as either Trade and Quote (TAQ) data, or *Open-High-Low-Close* (OHLC) data.

Trade and Quote (TAQ) data contains intraday *trades* and *quotes* on exchange-traded stocks and futures.

TAQ data is often called *tick data*, with a *tick* being a row of data containing new *trades* or *quotes*.

The TAQ data is spaced irregularly in time, with data recorded each time a new trade or quote arrives.

Each row of TAQ data may contain the quote and trade prices, and the corresponding quote size or trade volume: *Bid.Price*, *Bid.Size*, *Ask.Price*, *Ask.Size*, *Trade.Price*, *Volume*.

TAQ data is often split into *trade* data and *quote* data.

```
> # Load package HighFreq
> library(HighFreq)
> # Or load the high frequency data file directly:
> # symbolv <- load("/Users/jerzy/Develop/R/HighFreq/data/hf_data.R")
> head(HighFreq::SPY_TAQ)
> head(HighFreq::SPY)
> tail(HighFreq::SPY)
```

Downloading TAQ Data From WRDS

TAQ data can be downloaded from the *WRDS TAQ* web page.

The TAQ data are at millisecond frequency, and are *consolidated* (combined) from the New York Stock Exchange NYSE and other exchanges.

The *WRDS TAQ* web page provides separately *trades* data and separately *quotes* data.

Wharton RESEARCH DATA SERVICES

Search WRDS

Home / Get Data / TAQ / Millisecond Trade and Quote / TAQ - Millisecond Consolidated Trades

TAQ

Millisecond Trade and Quote

Consolidated Quotes

Consolidated Trades

TAQ Millisecond Tools

Query Form Variable Descriptions Manuals and Overviews FAQs Dataset List

TAQ - Millisecond Consolidated Trades

Note to TAQ users: Missing trades in the April 5, 2012 Consolidated Trades dataset.

There are roughly 65,000 missing trade records for tickers with initial letters M through R from 08:42 through 10:02:50 on April 5, 2012. NYSE has reported to WRDS that NASDAQ was not disseminating trade reports during that time period. The actual number of trade records (for "M" through "R") during that time from NASDAQ (exchange code "Q") was 2,232, but the average during the rest of the week was 65,274. The missing trades amount to about 12% of total trades in those equities.

WRDS will provide an update as soon as the files are available.

You have 1 saved query for this dataset.

Step 1: Choose your date range.

I would like data from Mar 18 2020 to Mar 18 2020

Step 2: What Time Range

Filter observations by time range

Beginning 00 30 00 Ending 18 00 00

Step 3: Apply your company codes.

CRISDOL (past partner)

Select an option for entering company codes

☒ AAPL ☐ Code List Name

Please enter Company codes separated by a space.
Example: WMT MFT DELL COB LOOKUP

☐ Browse ☐ No file selected

Reading TAQ Data From .csv Files

Trade and Quote (TAQ) data stored in .csv files can be very large, so it's better to read it using the function `data.table::fread()` which is much faster than the function `read.csv()`.

Each *trade* or *quote* contributes a *tick* (row) of data, and the number of ticks can be very large (hundred of thousands per day, or more).

The function `strptime()` coerces character strings representing the date and time into POSIXlt *date-time* objects.

The argument `format="%H:%M:%OS"` allows the parsing of fractional seconds, for example "15:59:59.989847074".

The function `as.POSIXct()` coerces objects into POSIXct *date-time* objects, with a numeric value representing the *moment of time* in seconds.

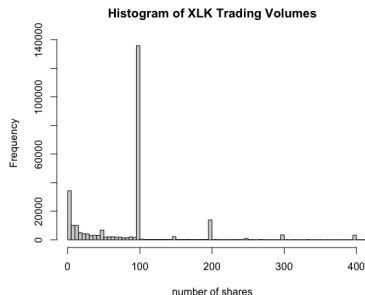
```
> library(HighFreq)
> # Read TAQ trade data from csv file
> taq <- data.table::fread(file="/Users/jerzy/Develop/lecture_slides/taq.csv")
> # Inspect the TAQ data in data.table format
> taq
> class(taq)
> colnames(taq)
> sapply(taq, class)
> symboln <- taq$SYM_ROOT[1]
> # Create date-time index
> datev <- paste(taq$DATE, taq$TIME_M)
> # Coerce date-time index to POSIXlt
> datev <- strptime(datev, "%Y%m%d %H:%M:%OS")
> class(datev)
> # Display more significant digits
> # options("digits")
> options(digits=20, digits.secs=10)
> last(datev)
> unclass(last(datev))
> as.numeric(last(datev))
> # Coerce date-time index to POSIXct
> datev <- as.POSIXct(datev)
> class(datev)
> last(datev)
> unclass(last(datev))
> as.numeric(last(datev))
> # Calculate the number of seconds
> as.numeric(last(datev)) - as.numeric(first(datev))
> # Calculate the number of ticks per second
> NROW(taq)/(6.5*3600)
> # Select TAQ data columns
> taq <- taq[, .(price=PRICE, volume=SIZE)]
```

Trading Volumes in High Frequency Data

The trading volumes represent the number of shares traded at a given price.

The histogram of the trading volumes shows that the highest frequencies of trades are for *round lots* - trades that are multiples of 100 shares.

There are also significant frequencies for *odd lots*, with small volumes of less than 100 shares.



```
> # Coerce trade ticks to xts series
> xlk <- xts::xts(taq[, .(price, volume)], datev)
> colnames(xlk) <- c("price", "volume")
> save(xlk, file="/Users/jerzy/Develop/data/xlk_tick_trades_2020031")
> # Plot histogram of the trading volumes
> hist(xlk$volume, main="Histogram of XLK Trading Volumes",
+      breaks=1e5, xlim=c(1, 400), xlab="number of shares")
```


Microstructure Noise in High Frequency Data

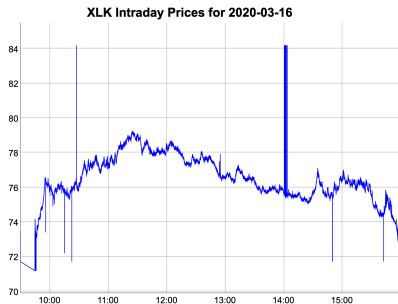
High frequency data contains *microstructure noise* in the form of *price spikes* and the *bid-ask bounce*.

Price spikes are single ticks with prices far away from the average.

Price spikes are often caused by data collection errors, but sometimes they represent actual trades with very large lot (trade) sizes.

The *bid-ask bounce* is the bouncing of traded prices between the bid and ask prices.

The *bid-ask bounce* creates an illusion of rapidly changing prices, while in reality the mid price is unchanged.



```
> # Plot dygraph
> dygraphs::dygraph(xlk$price, main="XLK Intraday Prices for 2020-03-16")
+ dyOptions(colors="blue", strokeWidth=1)
> # Plot in x11 window
> x11(width=6, height=5)
> quantmod::chart_Series(x=xlk$price, name="XLK Intraday Prices for 2020-03-16")
```

The Bid-ask Bounce of High Frequency Prices

The *bid-ask bounce* is the bouncing of traded prices between the bid and ask prices.

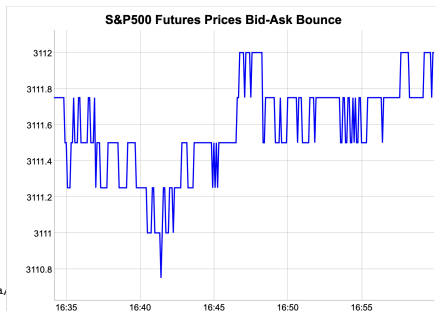
The *bid-ask bounce* is prominent at very high frequency time scales or in periods of low volatility.

The *bid-ask bounce* creates an illusion of rapidly changing prices, while in fact the mid price is constant.

The *bid-ask bounce* inflates the estimates of realized volatility, above the actual volatility.

The *bid-ask bounce* creates the appearance of mean reversion (negative autocorrelation), that isn't tradeable for most traders.

```
> pricev <- read.zoo(file="/Users/jerzy/Develop/lecture_slides/data,
+   header=TRUE, sep=",")
> pricev <- as.xts(pricev)
> dygraphs::dygraph(pricev$Close,
+   main="S&P500 Futures Prices Bid-Ask Bounce") %>%
+   dyOptions(colors="blue", strokeWidth=2)
```



Price Spikes And Trading Volumes in High Frequency Data

The number of the *price spikes* depends on the level of trading volumes, with the number decreasing with higher trading volumes.

The number of price spikes is lower for trade prices with larger trading volumes.



```
> # Plot dygraph of trade prices of at least 100 shares
> dygraphs::dygraph(xlk$price[xlk$volume >= 100, ],
+   main="XLK Prices for Trades of At Least 100 Shares") %>%
+   dyOptions(colors="blue", strokeWidth=1)
```

Removing Odd Lot Trades From TAQ Data

The trading volumes represent the number of shares traded at a given price.

The histogram of the trading volumes shows that the highest frequencies of trades are for *round lots* - trades that are multiples of 100 shares.

There are also significant frequencies for *odd lots*, with small volumes of less than 100 shares.

The *odd lot* ticks are often removed to reduce the size of the TAQ data.

Selecting only the large lot trades reduces microstructure noise (price spikes, bid-ask bounce) in high frequency data.

XLK Prices for Trades of At Least 100 Shares



```
> # Select the large trade lots of at least 100 shares
> dim(taq)
> tickb <- taq[taq$volume >= 100]
> dim(tickb)
> # Number of large lot ticks per second
> NROW(tickb)/(6.5*3600)
> # Plot histogram of the trading volumes
> hist(tickb$volume, main="Histogram of XLK Trading Volumes",
+      breaks=100000, xlim=c(1, 400), xlab="number of shares")
> # Save trade ticks with large lots
> data.table::fwrite(tickb, file="/Users/jerzy/Develop/data/xlk_tick_trades_2020-03-16_biglots.csv",
+                    col.names=c("price", "volume"))
```

```
> # Plot dygraph of the large lots
> dygraphs::dygraph(xlkb$price,
+   main="XLK Prices for Trades of At Least 100 Shares") %>%
+   dyOptions(colors="blue", strokeWidth=1)
> # Plot the large lots
> x11(width=6, height=5)
> quantmod::chart_Series(x=xlkb$price,
+   name="XLK Trade Ticks for 2020-03-16 (large lots only)")
```

Aggregating TAQ Data to OHLC

The *data table* columns can be *aggregated* over categories (factors) defined by one or more columns passed to the "by" argument.

Multiple *data table* columns can be referenced by passing a list of names specified by the dot `.` operator.

The function `round.POSIXt()` rounds date-time objects to seconds, minutes, hours, days, months or years.

The function `as.POSIXct()` coerces objects to class `POSIXct`.

```
> # Round time index to seconds
> tick[, zoo::index := as.POSIXct(round.POSIXt(index, "secs"))]
> # Aggregate to OHLC by seconds
> ohlc <- tick[, .(open=first(price), high=max(price), low=min(price), close=last(price)), by=index)
> # Round time index to minutes
> tick[, zoo::index := as.POSIXct(round.POSIXt(index, "mins"))]
> # Aggregate to OHLC by minutes
> ohlc <- tick[, .(open=first(price), high=max(price), low=min(price), close=last(price), volume=sum(volume)), by=index)
```



```
> # Coerce OHLC prices to xts
> ohlc <- xts::xts(ohlc[, -"index"], ohlc$index)
> # Plot dygraph of the OHLC prices
> dygraphs::dygraph(ohlc[, -5], main="XLK Trade Ticks for 2020-03-16",
+   dyCandlestick())
> # Plot the OHLC prices
> x11(width=6, height=5)
> quantmod::chart_Series(x=ohlc, TA="add_Vo()",
+   name="XLK Trade Ticks for 2020-03-16 (OHLC)")
```

Open-High-Low-Close (OHLC) Data

Open-High-Low-Close (OHLC) data contains intraday trade prices and trade volumes.

OHLC data is evenly spaced in time, with each row containing the *Open*, *High*, *Low*, *Close* prices, and the trade *Volume*, recorded over the past time interval (called a *bar* of data).

The *Open* and *Close* prices are the first and last trade prices recorded in the time bar.

The *High* and *Low* prices are the highest and lowest trade prices recorded in the time bar.

The *Volume* is the total trading volume recorded in the time bar.

The *OHLC* data format provides a way of efficiently compressing *TAQ* data, while preserving information about price levels, volatility (range), and trading volumes.

In addition, evenly spaced *OHLC* data allows for easier analysis of multiple time series, since the prices for different assets are given at the same moments in time.

```
> # Load package HighFreq
> library(HighFreq)
> head(HighFreq::SPY)
```

		SPY.Open	SPY.High	SPY.Low	SPY.Close	SPY.Volume
2008-01-02	09:31:00	147	147	147	147	591203
2008-01-02	09:32:00	147	147	147	147	385457
2008-01-02	09:33:00	147	147	147	147	343700
2008-01-02	09:34:00	147	147	147	147	863418
2008-01-02	09:35:00	147	147	147	147	457500
2008-01-02	09:36:00	147	147	147	147	416708

Plotting High Frequency OHLC Data

Aggregating high frequency *TAQ* data into *OHLC* format with lower periodicity allows for data compression while maintaining some information about volatility.

```
> # Load package HighFreq
> library(HighFreq)
> # Define symbol
> symboln <- "SPY"
> # Load OHLC data
> dirout <- "/Users/jerzy/Develop/data/minutes/"
> symboln <- load(file.path(dirout, paste0(symboln, ".RData")))
> interval <- "2013-11-11 09:30:00/2013-11-11 10:30:00"
> chart_Series(SPY[interval], name=symboln)
```

The package *HighFreq* contains both *TAQ* data and *Open-High-Low-Close (OHLC)* data.

If you are not able to install package *HighFreq* then download the file *hf_data.RData* from the NYU share drive and load it.



Package *HighFreq* for Managing High Frequency Data

The package *HighFreq* contains functions for managing high frequency time series data, such as:

- converting *TAQ* data to *OHLC* format,
- chaining and joining time series,
- scrubbing bad data,
- managing time zones and aligning time indices,
- aggregating data to lower frequency (periodicity),
- calculating rolling aggregations (VWAP, Hurst exponent, etc.),
- calculating seasonality aggregations,
- estimating volatility, skewness, and higher moments,

```
> # Install package HighFreq from github
> devtools::install_github(repo="algoquant/HighFreq")
> # Load package HighFreq
> library(HighFreq)
> # Get documentation for package HighFreq
> # Get short description
> packageDescription(HighFreq)
> # Load help page
> help(package=HighFreq)
> # List all datasets in HighFreq
> data(package=HighFreq)
> # List all objects in HighFreq
> ls("package:HighFreq")
> # Remove HighFreq from search path
> detach("package:HighFreq")
```


Datasets in Package *HighFreq*

The package *HighFreq* contains several high frequency time series, in *xts* format, stored in a file called `hf_data.RData`:

- a time series called `SPY.TAQ`, containing a single day of *TAQ* data for the *SPY* ETF.
- three time series called `SPY`, `TLT`, and `VXX`, containing intraday 1-minute *OHLC* price bars for the *SPY*, *TLT*, and *VXX* ETFs.

Even after the *HighFreq* package is loaded, its datasets aren't loaded into the workspace, so they aren't listed in the workspace.

That's because the datasets in package *HighFreq* are set up for *lazy loading*, which means they can be called as if they were loaded, even though they're not loaded into the workspace.

The datasets in package *HighFreq* can be loaded into the workspace using the function `data()`.

The data is set up for *lazy loading*, so it doesn't require calling `data(hf_data)` to load it into the workspace before calling it.

```
> # Load package HighFreq
> library(HighFreq)
> # You can see SPY when listing objects in HighFreq
> ls("package:HighFreq")
> # You can see SPY when listing datasets in HighFreq
> data(package=HighFreq)
> # But the SPY dataset isn't listed in the workspace
> ls()
> # HighFreq datasets are lazy loaded and available when needed
> head(HighFreq::SPY)
> # Load all the datasets in package HighFreq
> data(hf_data)
> # HighFreq datasets are now loaded and in the workspace
> head(HighFreq::SPY)
```

Distribution of High Frequency Returns

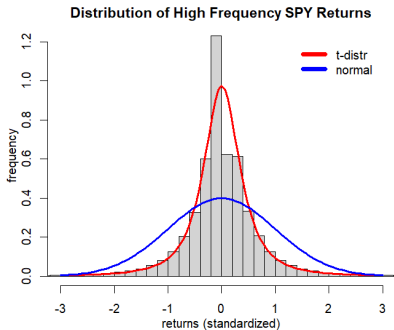
High frequency returns exhibit *large negative skewness* and *very large kurtosis* (leptokurtosis), or fat tails.

Student's *t-distribution* has fat tails, so it fits high frequency returns much better than the normal distribution.

The function `fitdistr()` from package *MASS* fits a univariate distribution into a sample of data, by performing *maximum likelihood* optimization.

The function `hist()` calculates and plots a histogram, and returns its data *invisibly*.

```
> # Calculate SPY percentage returns
> ohlc <- HighFreq::SPY
> nrow <- NROW(ohlc)
> closep <- log(quantmod::Cl(ohlc))
> retp <- rutils::diffit(closep)
> colnames(retp) <- "SPY"
> # Standardize raw returns to make later comparisons
> retp <- (retp - mean(retp))/sd(retp)
> # Calculate moments and perform normality test
> sapply(c(var=2, skew=3, kurt=4), function(x) sum(retp^x)/nrow)
> tseries::jarque.bera.test(retp)
> # Fit SPY returns using MASS::fitdistr()
> optiml <- MASS::fitdistr(retp, densfun="t", df=2)
> loc <- optiml$estimate[1]
> scalev <- optiml$estimate[2]
```



```
> # Plot histogram of SPY returns
> histp <- hist(retp, col="lightgrey", mgp=c(2, 1, 0),
+ xlab="returns (standardized)", ylab="frequency", xlim=c(-3, 3),
+ breaks=1e3, freq=FALSE, main="Distribution of High Frequency SPY")
> # lines(density(retp, bw=0.2), lwd=3, col="blue")
> # Plot t-distribution function
> curve(expr=dt((x-loc)/scalev, df=2)/scalev,
+ type="l", lwd=3, col="red", add=TRUE)
> # Plot the Normal probability distribution
> curve(expr=dnorm(x, mean=mean(retp),
+ sd=sd(retp)), add=TRUE, lwd=3, col="blue")
> # Add legend
> legend("topright", inset=0.05, bty="n",
+ leg=c("t-distr", "normal"), y.intersp=0.5,
+ lwd=6, lty=1, col=c("red", "blue"))
```

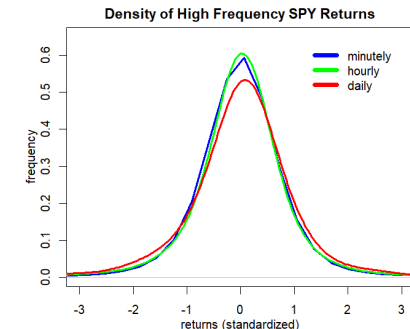
Distribution of Aggregated High Frequency Returns

The distribution of returns depends on the sampling frequency.

High frequency returns aggregated to a lower periodicity become less negatively skewed and less fat tailed, and closer to the normal distribution.

The function `xts::to.period()` converts a time series to a lower periodicity (for example from hourly to daily periodicity).

```
> # Hourly SPY percentage returns
> closep <- log(Cl(xts::to.period(x=ohlcv, period="hours")))
> retsh <- rutils::diffit(closep)
> retsh <- (retsh - mean(retsh))/sd(retsh)
> # Daily SPY percentage returns
> closep <- log(Cl(xts::to.period(x=ohlcv, period="days")))
> retld <- rutils::diffit(closep)
> retld <- (retld - mean(retld))/sd(retld)
> # Calculate moments
> sapply(list(minutely=retp, hourly=retsh, daily=retld),
+ function(rets) {sapply(c(var=2, skew=3, kurt=4),
+ function(x) mean(rets^x))
+ }) ## end sapply
```



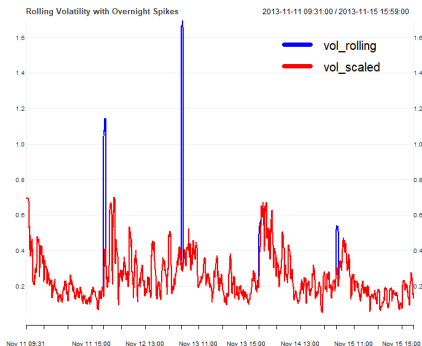
```
> # Plot densities of SPY returns
> plot(density(retp, bw=0.4), xlim=c(-3, 3),
+      lwd=3, mgp=c(2, 1, 0), col="blue",
+      xlab="returns (standardized)", ylab="frequency",
+      main="Density of High Frequency SPY Returns")
> lines(density(retsh, bw=0.4), lwd=3, col="green")
> lines(density(retld, bw=0.4), lwd=3, col="red")
> # Add legend
> legend("topright", inset=0.05, bty="n",
+      leg=c("minutely", "hourly", "daily"), y.intersp=0.5,
+      lwd=6, lty=1, col=c("blue", "green", "red"))
```

Estimating Rolling Volatility of High Frequency Returns

The volatility of high frequency returns can be inflated by large overnight returns.

The large overnight returns can be scaled down by dividing them by the overnight time interval.

```
> # Calculate rolling volatility of SPY returns
> ret2013 <- retp["2013-11-11/2013-11-15"]
> # Calculate rolling volatility
> lookb <- 11 ## Look-back interval
> endd <- seq_along(ret2013)
> startp <- c(rep_len(1, lookb),
+   endd[1:(NROW(endd)-lookb)])
> endd[endd < lookb] <- lookb
> vol_rolling <- sapply(seq_along(endd),
+   function(it) sd(ret2013[startp[it]:endd[it]]))
> vol_rolling <- xts::xts(vol_rolling, zoo::index(ret2013))
> # Extract time intervals of SPY returns
> indeks <- c(60, diff(xts::index(ret2013)))
> head(indeks)
> table(indeks)
> # Scale SPY returns by time intervals
> ret2013 <- 60*ret2013/indeks
> # Calculate scaled rolling volatility
> vol_scaled <- sapply(seq_along(endd),
+   function(it) sd(ret2013[startp[it]:endd[it]]))
> vol_rolling <- cbind(vol_rolling, vol_scaled)
> vol_rolling <- na.omit(vol_rolling)
> sum(is.na(vol_rolling))
> sapply(vol_rolling, range)
```



```
> # Plot rolling volatility
> x11(width=6, height=5)
> themev <- chart_theme()
> themev$col$line.col <- c("blue", "red")
> chart_Series(vol_rolling, theme=themev,
+   name="Rolling Volatility with Overnight Spikes")
> legend("topright", legend=colnames(vol_rolling),
+   inset=0.1, bg="white", lty=1, lwd=6, y.intersp=0.5,
+   col=themev$col$line.col, bty="n")
```

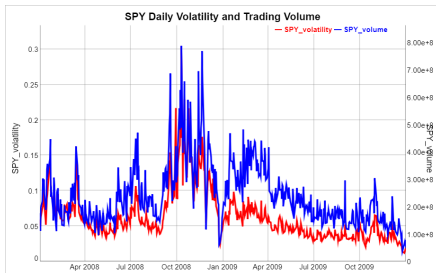
Daily Volume and Volatility of High Frequency Returns

Trading volumes typically rise together with market price volatility.

The function `apply.daily()` from package `xts` applies functions to time series over daily periods.

The function `calc_var_ohlc()` from package `HighFreq` calculates the variance of an *OHLC* time series using range estimators.

```
> # Volatility of SPY
> sqrt(HighFreq::calc_var_ohlc(ohlc))
> # Daily SPY volatility and volume
> volatd <- sqrt(xts::apply.daily(ohlc, FUN=calc_var_ohlc))
> colnames(volatd) <- ("SPY_volatility")
> volumv <- quantmod::Vo(ohlc)
> volumd <- xts::apply.daily(volumv, FUN=sum)
> colnames(volumd) <- ("SPY_volume")
> # Plot SPY volatility and volume
> datav <- cbind(volatd, volumd)["2008/2009"]
> colv <- colnames(datav)
> dygraphs::dygraph(datav,
+   main="SPY Daily Volatility and Trading Volume") %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", col="red", strokeWidth=3) %>%
+   dySeries(name=colv[2], axis="y2", col="blue", strokeWidth=3)
```



Beta of Volume vs Volatility of High Frequency Returns

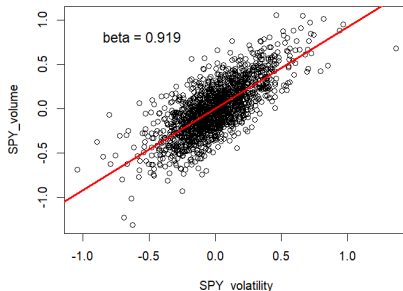
As a general empirical rule, the *trading volume* v in a given time period is roughly proportional to the *volatility* of the returns σ : $v \propto \sigma$.

The regression of the *log trading volume* versus the *log volatility* fails the *Durbin-Watson test* for the autocorrelation of residuals.

But the regression of the *differences* passes the *Durbin-Watson test*.

```
> # Regress log of daily volume vs volatility
> datav <- log(cbind(volumd, volatd))
> colv <- colnames(datav)
> dframe <- as.data.frame(datav)
> formulav <- as.formula(paste(colv, collapse="~"))
> regmod <- lm(formulav, data=dframe)
> # Durbin-Watson test for autocorrelation of residuals
> lmtest::dwtest(regmod)
> # Regress diff log of daily volume vs volatility
> dframe <- as.data.frame(rutils::diffit(datav))
> regmod <- lm(formulav, data=dframe)
> lmtest::dwtest(regmod)
> summary(regmod)
> plot(formulav, data=dframe, main="SPY Daily Trading Volume vs Volatility (log scale)")
> abline(regmod, lwd=3, col="red")
> mtext(paste("beta =", round(coef(regmod)[2], 3)), cex=1.2, lwd=3, side=2, las=2, adj=(-0.5), padj=(-7))
```

SPY Daily Trading Volume vs Volatility (log scale)

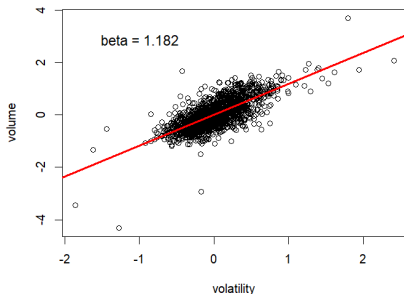


Hourly Trading Volume vs Volatility

Hourly aggregations of high frequency data also support the rule that the *trading volume* is roughly proportional to the *volatility* of the returns: $v \propto \sigma$.

```
> # 60 minutes of data in lookb interval
> lookb <- 60 ## Look-back interval
> vol2013 <- volumv["2013"]
> ret2013 <- retp["2013"]
> # Define end points with beginning stub
> nrow <- NROW(ret2013)
> nagg <- nrow %/% lookb
> endd <- nrow-lookb*nagg + (0:nagg)*lookb
> startp <- c(1, endd[1:(NROW(endd)-1)])
> # Calculate SPY volatility and volume
> datav <- sapply(seq_along(endd), function(it) {
+   endp <- startp[it]:endd[it]
+   c(volume=sum(vol2013[endp]),
+     volatility=sd(ret2013[endp]))
+ }) ## end sapply
> datav <- t(datav)
> datav <- rutils::diffit(log(datav))
> dframe <- as.data.frame(datav)
```

SPY Hourly Trading Volume vs Volatility (log scale)



```
> formulav <- as.formula(paste(colnames(datav), collapse=""))
> regmod <- lm(formulav, data=dframe)
> lmtest::dwtest(regmod)
> summary(regmod)
> plot(formulav, data=dframe,
+   main="SPY Hourly Trading Volume vs Volatility (log scale)")
> abline(regmod, lwd=3, col="red")
> mtext(paste("beta =", round(coef(regmod)[2], 3)), cex=1.2, lwd=3,
```

High Frequency Returns in Trading Time

The *trading time* (volume clock) is the time measured by the level of *trading volume*, with the *volume clock* running faster in periods of higher *trading volume*.

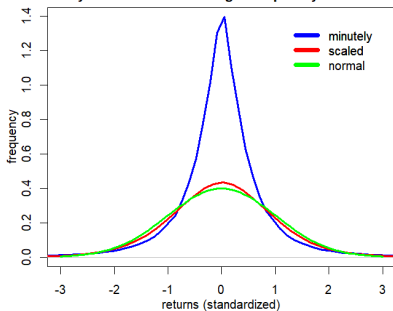
The time-dependent volatility of high frequency returns (*heteroskedasticity*) produces their *leptokurtosis* (large kurtosis, or fat tails).

The returns can be divided by the *square root of the trading volumes* to obtain scaled returns over equal trading volumes.

But the returns should not be divided by very small volumes below a certain threshold.

The scaled returns have a smaller *skewness* and *kurtosis*, and they also have even higher autocorrelations than unscaled returns.

Density of Volume-scaled High Frequency SPY Returns



```
> # Scale returns using volume (volume clock)
> retsc <- ifelse(volumv > 1e4, retp/sqrt(volumv), 0)
> retsc <- retsc/sd(retsc)
> # Calculate moments of scaled returns
> nrow <- NROW(retp)
> sapply(list(retp=retp, retsc=retsc),
+   function(rets) {sapply(c(skew=3, kurt=4),
+     function(x) sum((rets/sd(rets))^x)/nrow)
+ }) ## end sapply
```

```
> x11(width=6, height=5)
> par(mar=c(3, 3, 2, 1), oma=c(1, 1, 1, 1))
> # Plot densities of SPY returns
> plot(density(retp), xlim=c(-3, 3),
+   lwd=3, mgp=c(2, 1, 0), col="blue",
+   xlab="returns (standardized)", ylab="frequency",
+   main="Density of Volume-scaled High Frequency SPY Returns")
> lines(density(retsc, bw=0.4), lwd=3, col="red")
> curve(expr=dnorm, add=TRUE, lwd=3, col="green")
> # Add legend
> legend("topright", inset=0.05, bty="n", y.intersp=0.5,
+   leg=c("minutely", "scaled", "normal"),
+   lwd=6, lty=1, col=c("blue", "red", "green"))
```


Autocorrelations of High Frequency Returns

The *Ljung-Box* test, tests if the autocorrelations of a time series are *statistically significant*.

The *null hypothesis* of the *Ljung-Box* test is that the autocorrelations are equal to zero.

The *Ljung-Box* statistic is small for time series that have *statistically insignificant* autocorrelations.

The function `Box.test()` calculates the *Ljung-Box* test and returns the test statistic and its p-value.

For *minutely SPY* returns, the *Ljung-Box* statistic is large and its *p*-value is very small, so we can conclude that *minutely SPY* returns have statistically significant autocorrelations.

For *scaled minutely SPY* returns, the *Ljung-Box* statistic is even larger, so its autocorrelations are even more statistically significant.

SPY returns aggregated to longer time intervals are less autocorrelated.

```
> # Ljung-Box test for minutely SPY returns
> Box.test(retp, lag=10, type="Ljung")
> # Ljung-Box test for daily SPY returns
> Box.test(retd, lag=10, type="Ljung")
> # Ljung-Box test statistics for scaled SPY returns
> sapply(list(retp=retp, retsc=retsc),
+   function(rets) {
+     Box.test(rets, lag=10, type="Ljung")$statistic
+ }) ## end sapply
> # Ljung-Box test statistics for aggregated SPY returns
> sapply(list(minutely=retp, hourly=retsh, daily=retd),
+   function(rets) {
+     Box.test(rets, lag=10, type="Ljung")$statistic
+ }) ## end sapply
```

The level of the autocorrelations depends on the sampling frequency, with higher frequency returns having more significant negative autocorrelations.

As the returns are aggregated to a lower periodicity, they become less autocorrelated, with daily returns having almost insignificant autocorrelations.

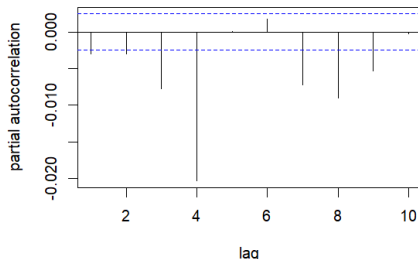
Partial Autocorrelations of High Frequency Returns

High frequency minutely *SPY* returns have statistically significant negative autocorrelations.

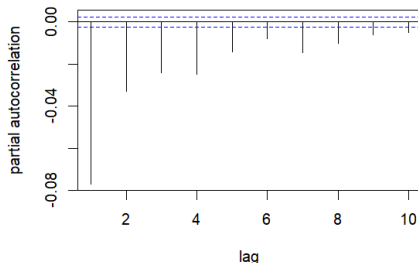
SPY returns *scaled* by the trading volumes have even more significant negative autocorrelations.

```
> # Set plot parameters
> x11(width=6, height=8)
> par(mar=c(4, 4, 2, 1), oma=c(0, 0, 0, 0))
> layout(matrix(c(1, 2), ncol=1), widths=c(6, 6), heights=c(4, 4))
> # Plot the partial autocorrelations of minutely SPY returns
> pacf1 <- pacf(as.numeric(retp), lag=10,
+   xlab="lag", ylab="partial autocorrelation", main="")
> title("Partial Autocorrelations of Minutely SPY Returns", line=1)
> # Plot the partial autocorrelations of scaled SPY returns
> pacfs <- pacf(as.numeric(retsc), lag=10,
+   xlab="lag", ylab="partial autocorrelation", main="")
> title("Partial Autocorrelations of Scaled SPY Returns", line=1)
> # Calculate the sums of partial autocorrelations
> sum(pacf1$acf)
> sum(pacfs$acf)
```

Partial Autocorrelations of Minutely SPY Return



Partial Autocorrelations of Scaled SPY Return



Market Liquidity, Trading Volume and Volatility

Market illiquidity is defined as the market price impact resulting from supply-demand imbalance.

Market liquidity $\mathcal{L}$ is proportional to the square root of the trading volume v divided by the price volatility σ :

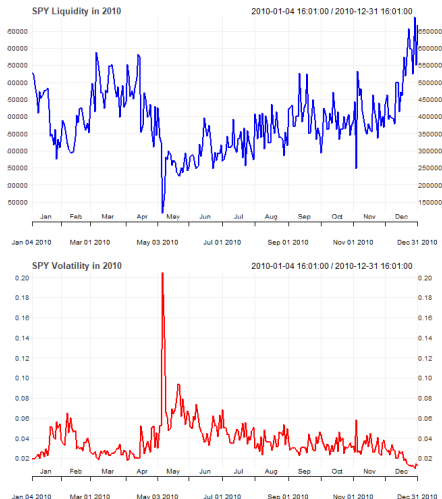
$$\mathcal{L} \sim \frac{\sqrt{v}}{\sigma}$$

Market illiquidity spiked during the May 6, 2010 *flash crash*.

Research suggests that market crashes are caused by declining market liquidity:

Donier et al., Why Do Markets Crash?

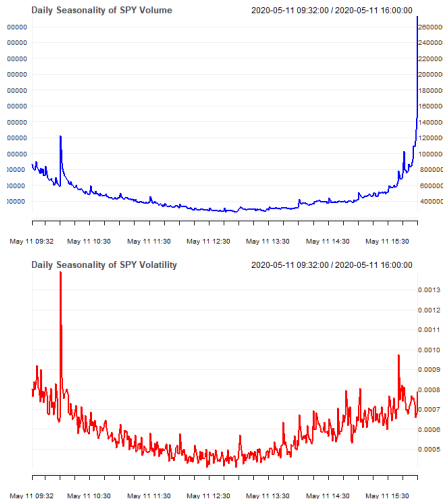
```
> # Calculate market illiquidity
> liquidv <- sqrt(volumd)/volatd
> # Plot market illiquidity
> x11(width=6, height=7) ; par(mfrow=c(2, 1))
> themev <- chart_theme()
> themev$col$line.col <- c("blue")
> chart_Series(liquidv["2010"], theme=themev,
+   name="SPY Liquidity in 2010", plot=FALSE)
> themev$col$line.col <- c("red")
> chart_Series(volatd["2010"],
+   theme=themev, name="SPY Volatility in 2010")
```



Intraday Seasonality of Volume and Volatility

The volatility and trading volumes are typically higher at the beginning and end of the trading sessions.

```
> # Calculate intraday time index with hours and minutes
> datev <- format(zoo::index(retp), "%H:%M")
> # Aggregate the mean volume
> volumagg <- tapply(X=volumv, INDEX=datev, FUN=mean)
> volumagg <- drop(volumagg)
> # Aggregate the mean volatility
> volagg <- tapply(X=retp^2, INDEX=datev, FUN=mean)
> volagg <- sqrt(drop(volagg))
> # Coerce to xts
> datev <- as.POSIXct(paste(Sys.Date(), names(volumagg)))
> volumagg <- xts::xts(volumagg, datev)
> volagg <- xts::xts(volagg, datev)
> # Plot seasonality of volume and volatility
> x11(width=6, height=7) ; par(mfrow=c(2, 1))
> themev <- chart_theme()
> themev$col$line.col <- c("blue")
> chart_Series(volumagg[c(-1, -NROW(volumagg))], theme=themev,
+   name="Intraday Seasonality of SPY Volume", plot=FALSE)
> themev$col$line.col <- c("red")
> chart_Series(volagg[c(-1, -NROW(volagg))], theme=themev,
+   name="Intraday Seasonality of SPY Volatility")
```

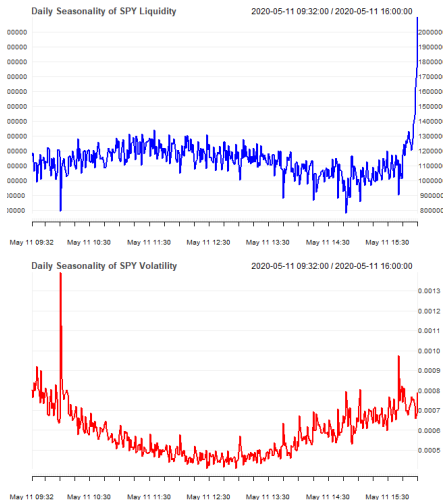


Intraday Seasonality of Liquidity and Volatility

Market liquidity is typically the highest at the end of the trading session, and the lowest at the beginning.

The end of day spike in trading volumes and liquidity is driven by computer-driven investors liquidating their positions.

```
> # Calculate market liquidity
> liquidv <- sqrt(volumagg)/volagg
> # Plot intraday seasonality of market liquidity
> x11(width=6, height=7) ; par(mfrow=c(2, 1))
> themev <- chart_theme()
> themev$col$line.col <- c("blue")
> chart_Series(liquidv[c(-1, -NROW(liquidv))], theme=themev,
+   name="Intraday Seasonality of SPY Liquidity", plot=FALSE)
> themev$col$line.col <- c("red")
> chart_Series(volagg[c(-1, -NROW(volagg))], theme=themev,
+   name="Intraday Seasonality of SPY Volatility")
```



Homework Assignment

Required

- Study all the lecture slides in *FRE6871_Lecture_5.pdf*, and run all the code in *FRE6871_Lecture_5.R*

Recommended

- Read about *PCA* in:
pca-handout.pdf
pcaTutorial.pdf
- Read about *optimization methods*:
Bolker Optimization Methods.pdf, *Yollin Optimization.pdf*, *Boudt DEoptim Large Portfolio Optimization.pdf*.